

# Guardrails and Agent Observability

CloudWatch, Tracing, Logging, Metrics and Evaluation

*Today's learning notes for quick project revision*

**Protect Inputs -> Run Agent -> Observe Journey -> Inspect Details -> Measure Quality -> Improve**

## **Main learning**

Authentication checks who the caller is. Cedar policies check what action is allowed. Guardrails check whether the content is safe. Traces show where execution went. Logs explain what happened and why. Evaluations measure whether the final agent behavior was good.

# 1. Guardrails: Purpose and Placement

|             |                                                                                                                                                                                 |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>What</b> | A guardrail is a content-safety check for user input, model output, tool input or tool output.                                                                                  |
| <b>Why</b>  | Models and users may accidentally introduce sensitive, unsafe or malicious content. That content should be stopped before it reaches downstream systems or customers.           |
| <b>How</b>  | A configured safeguard analyzes selected content and produces a confidence signal. A policy uses that signal to permit or forbid the action, and Gateway enforces the decision. |

## 1.1 Guardrail Used in This Project

- Gateway: my-gateway-secure
- Policy Engine: CustomerSupportPolicyEngine
- Target and tool: ProcessRefund\_\_process\_refund
- Field inspected: context.input.reason
- Sensitive category: EMAIL
- Effect: forbid the tool call when an email is detected above the configured threshold

The project used a tool-input guardrail. The original customer message was already seen by the model. The guardrail inspected the structured refund reason created by the model immediately before Gateway invoked the Refund Lambda.

**User message -> Model creates tool arguments -> Gateway intercepts tool call  
-> Guardrail checks reason -> Permit or block -> Lambda only if safe**

### Precise responsibility

Gateway is the enforcement boundary. The Policy Engine evaluates the rule. Bedrock Guardrails supplies the content-safety signal. Gateway blocks or allows the downstream tool call.

## 2. Refund Guardrail Example

### 2.1 Normal Refund Request

Customer request: “Please refund \$45 for order ORD-12345 because I received the wrong product.”

1. Runtime receives the authenticated request.
2. The Foundation Model identifies a refund intent.
3. The model selects ProcessRefund\_\_process\_refund.
4. The model creates order\_id = ORD-12345, amount = 45, and reason = Received the wrong product.
5. Runtime sends the tool call through the MCP client to Gateway.
6. Gateway validates the JWT.
7. The Cedar rule confirms that 45 is below the automated limit of 100.
8. The guardrail checks the reason and finds no email address.
9. Gateway invokes the Refund Lambda.
10. The result returns to Runtime and the model prepares the final response.

#### Result

The amount is allowed and the reason is safe, so the refund tool can execute.

### 2.2 Same Request With Sensitive Information

Customer request: “Please refund \$45 for order ORD-12345 because I received the wrong product. Contact me at alice@example.com.”

11. The model creates the refund reason containing alice@example.com.
12. The amount policy permits 45 because it is below 100.
13. The EMAIL guardrail detects the literal email address in context.input.reason.
14. The forbid condition overrides the amount permit.
15. Gateway blocks that specific tool invocation.
16. The Refund Lambda does not execute with the unsafe reason.

### 2.3 What the Agent Should Say

The agent should not always say “refund is impossible.” The refund may still be valid after unnecessary personal information is removed. A better response is: “The refund reason contains personal information that is not required for processing. I can retry using the general reason ‘wrong product received’ without including the email address.”

The model may safely retry with reason = Customer received the wrong product. If the amount remains below the limit and no email is present, the safe retry can reach Lambda.

#### Important

The guardrail does not decide whether an email address is correct or incorrect. It only detects that a literal email address is present in the protected tool input.

## 3. Where Guardrails Can Be Used

### 3.1 User-Input Guardrail

Checks the original user message before the model processes it. Useful for prompt injection, jailbreak attempts, secrets that should never enter the model, and restricted content.

### 3.2 Model-Output Guardrail

Checks the final model response before it reaches the user. Useful for preventing PII exposure, confidential company information, unsafe language or restricted content.

### 3.3 Tool-Input Guardrail

Checks structured arguments created by the model before a tool runs. This is the pattern used in the project for the refund reason. It is useful for preventing email addresses, passwords, card numbers or unsafe commands from reaching downstream systems.

### 3.4 Tool-Output Guardrail

Checks data returned by Lambda, an API or a database before the model uses it. It can stop secrets, another customer's PII or internal fields from reaching the agent response.

### 3.5 RAG and Agent-to-Agent Guardrails

- RAG: inspect retrieved documents for prompt injection, confidential data or unsafe external content.
- Multi-agent systems: inspect information passed from a research agent to an action agent.
- Code or shell agents: check unsafe instructions, but also rely on deterministic sandboxing and permissions.

#### **Defense in depth**

Guardrails do not replace JWT authentication, Cedar authorization, IAM permissions, input validation, encryption, sandboxing or human approval. Use several layers together.

## 4. Authentication, Policy and Guardrail

| Security question                 | Project mechanism                                |
|-----------------------------------|--------------------------------------------------|
| Who is calling?                   | Cognito JWT authentication                       |
| What action is allowed?           | Policy Engine with Cedar rules                   |
| Is the content safe?              | Bedrock Guardrails used inside policy evaluation |
| Who performs the business action? | Refund or Warranty Lambda                        |

### 4.1 Examples

- Expired JWT: authentication fails before tool access.
- \$500 refund: Cedar authorization denies the action because the amount exceeds the automated limit.
- \$45 refund containing alice@example.com: amount is permitted, but the guardrail forbids the unsafe tool input.
- \$45 refund with reason "wrong product received": authentication, authorization and guardrail checks pass.

**JWT validation -> Cedar action check -> Guardrail content check -> Lambda execution**

## 5. Amazon CloudWatch and Observability

|             |                                                                                                                                                                                             |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>What</b> | CloudWatch stores and displays operational telemetry such as logs, traces and metrics for Runtime, Gateway, tools and evaluations.                                                          |
| <b>Why</b>  | Agent failures can be operational or behavioral. A request may technically succeed while using the wrong tool or returning an incorrect answer. Developers need visibility into every step. |
| <b>How</b>  | AgentCore and OpenTelemetry instrumentation emit telemetry. CloudWatch stores and visualizes it through AgentCore GenAI Observability views.                                                |

### 5.1 Why CloudWatch Is Needed

- Find which component failed.
- Inspect Runtime, model, memory, MCP, Gateway and Lambda activity.
- Measure latency and error rates.
- View exact tool calls and policy/guardrail decisions when captured.
- Correlate one user session with its traces and logs.
- View evaluation trends and investigate low-quality interactions.

### 5.2 OpenTelemetry

OpenTelemetry is the standard instrumentation and telemetry format used to capture traces, logs and metrics. A simple analogy is: OpenTelemetry is the camera and sensor system; CloudWatch is the control room where the data is stored and viewed.

#### Operational question

CloudWatch helps answer: What happened, where did it happen, when did it happen, why did it fail, how long did it take, and how often is it happening?

## 6. Tracing and Spans

|             |                                                                                                                                                              |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>What</b> | A trace is the complete journey of one user request. A span is one operation inside that journey.                                                            |
| <b>Why</b>  | Agents cross many components. A single final error does not reveal whether the failure occurred in the model, memory, MCP client, Gateway, policy or Lambda. |
| <b>How</b>  | Each operation emits a span during execution. Related spans share a trace identity and form the complete request path.                                       |

### 6.1 Warranty Trace Example

17. Runtime receives the request.
18. Memory retrieves user context.
19. The Foundation Model processes the prompt.
20. The model selects the warranty tool.
21. The MCP client contacts Gateway.
22. Gateway validates authentication.
23. The Policy Engine checks authorization.
24. Gateway invokes the Warranty Lambda.
25. Lambda returns warranty status.
26. The model generates the final answer.
27. Runtime sends the response.

### 6.2 When Tracing Happens

Tracing occurs while the request is running. Spans are created as each step starts and ends. The complete trace is easiest to review after the request finishes and CloudWatch has indexed all spans. A failed request can still produce a trace showing where execution stopped.

#### Trace purpose

A trace helps locate where execution went and where it stopped. It also shows order, parent-child relationships and duration.

## 7. Logging

|             |                                                                                                                                                 |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>What</b> | Logs are detailed timestamped records produced by application code and AWS services.                                                            |
| <b>Why</b>  | After a trace identifies the failing component, developers need exact values, messages and error details to understand the cause.               |
| <b>How</b>  | Components write INFO, WARNING, ERROR or DEBUG events containing context such as tool names, arguments, policy decisions and backend responses. |

### 7.1 Refund Logging Example

- Runtime received request.
- User authenticated successfully.
- Model selected ProcessRefund\_\_process\_refund.
- Tool arguments contained order ID, amount and reason.
- Amount policy permitted or denied the request.
- Sensitive-information check passed or detected EMAIL.
- Refund Lambda executed or was skipped.
- Refund completed successfully or returned an error.

### 7.2 Example Failure Details

- JWT expired.
- Refund amount 500 failed the amount < 100 rule.
- BlockSensitiveRefundReasons detected an email.
- Lambda returned invalid JSON.
- Required tool argument was missing.

#### Log purpose

Logs explain the exact event, values and reason behind success or failure.



## 8. Trace vs Log vs Metric

| Trace                                 | Log                                         |
|---------------------------------------|---------------------------------------------|
| Complete request journey              | Detailed event or message                   |
| Shows component order and timing      | Shows exact values, warnings and errors     |
| Helps locate where the issue occurred | Helps explain why the issue occurred        |
| Contains spans                        | May be emitted inside any span or component |

### 8.1 Metrics

Metrics are numerical measurements over time, such as request count, error rate, average latency, model duration, Gateway failures, Lambda duration and token usage. Metrics tell how often and how much; traces and logs explain individual requests.

### 8.2 Easy Analogy

- Trace = Route map of a train journey.
- Span = One station or stage in that journey.
- Log = Detailed event register at a station.
- Metric = Average delay, failure count or number of passengers over time.

#### Key sentence

Trace helps locate the problem. Logs help explain the problem. Metrics show frequency and performance trends.

## 9. Agent Evaluation in AWS

|             |                                                                                                                                                               |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>What</b> | Evaluation automatically scores the quality of agent behavior and responses using built-in or custom evaluators.                                              |
| <b>Why</b>  | A request can return successfully while still being incorrect, unhelpful or based on the wrong tool. Operational success is not the same as customer success. |
| <b>How</b>  | The evaluator reviews traces containing the user request, tool calls, tool results and final answer, then assigns scores and explanations.                    |

### 9.1 Evaluators Used in the Project

- Goal Success Rate: Did the agent actually solve the customer's problem?
- Correctness: Was the answer factually accurate and supported by the available information?
- Tool Selection Accuracy: Did the model select the appropriate tool for the request?

### 9.2 Example

User asks for the warranty of PROD-001. If the agent selects `check_warranty` and returns the correct status, all scores may be high. If the agent selects `get_return_policy` or invents an expiration date, tool selection and correctness scores decrease even if the application returned a normal response.

### 9.3 Online vs On-Demand Evaluation

| Online Evaluation                      | On-Demand Evaluation                    |
|----------------------------------------|-----------------------------------------|
| Continuously samples new live sessions | Manually evaluates historical sessions  |
| Useful for production monitoring       | Useful for investigation and comparison |
| Tracks quality trends over time        | Runs over a chosen lookback period      |

### 9.4 LLM-as-a-Judge

A judge model reviews the interaction and assigns a score with reasoning. The score is useful for trends and investigation, but important business decisions should also use representative test datasets, human review and deterministic checks where possible.

## 10. How Observability and Evaluation Work Together

Suppose Tool Selection Accuracy falls to 60%. Evaluation tells you there is a quality problem. A trace shows that the model chose `get_return_policy` instead of `check_warranty`. Logs show the exact selected tool and arguments. Metrics show how often the issue occurs. You then improve the tool description or system prompt, generate new traffic and compare evaluation scores again.

**Evaluation detects quality issue -> Trace locates wrong step -> Logs explain exact cause -> Metrics show trend -> Prompt/tool fix -> Re-evaluate**

### 10.1 Debugging a \$500 Refund

- 28. Trace shows execution stopped at Policy Engine before Lambda.
- 29. Logs show amount = 500 and `refund_limit_policy` returned DENY.
- 30. Metrics show how many refund requests are being denied.
- 31. Evaluation checks whether the agent explained the denial helpfully and correctly.

### 10.2 Debugging an Email Guardrail Denial

- 32. Trace shows the guardrail/policy stage denied the first tool call.
- 33. Logs show the protected reason contained an email and identify the denying policy.
- 34. Trace may show a second safe retry without the email.
- 35. Evaluation checks whether the final response solved the user's goal without exposing sensitive information.

## 11. Tool Selection When Tools Overlap

If a local tool and a Lambda-backed MCP tool provide similar information, clearly separate their responsibilities. For example, `get_product_info` can provide static product details, while `check_warranty` provides authoritative current warranty status and expiration.

- Write specific, non-overlapping tool descriptions.
- Define which source is authoritative for each field.
- Remove unnecessary duplicate outputs.
- Use sequential calls when one tool provides the ID needed by another.
- Use Tool Selection Accuracy evaluation to verify behavior.

### 11.1 If Both Tools Perform Exactly the Same Task

Keep only one tool available whenever possible. Prefer the most authoritative, current, reliable and governed source. If the Lambda-backed tool represents the live company system, remove the duplicate local tool from the agent tool list.

Keep both only when one has a clearly defined fallback role. In that case, use the remote authoritative tool first and the local cached tool only after a failure. Important fallback behavior should be deterministic rather than left entirely to model choice.

#### **Best design principle**

One tool should have one clear responsibility. Duplicate tools increase confusion, latency, cost and inconsistent answers.

## 12. Final Revision Notes

- Guardrail checks whether protected content is safe.
- This project checked the refund tool reason for an EMAIL entity.
- Gateway blocks the unsafe tool invocation before Lambda executes.
- The agent may explain the issue or retry safely without the sensitive value.
- CloudWatch is the monitoring and observability platform.
- Tracing records the request journey while the agent runs.
- A span is one operation inside a trace.
- Logs contain detailed events, values, warnings and errors.
- Metrics show numerical performance over time.
- Evaluation measures goal success, correctness and tool selection quality.
- Trace tells where and when. Logs tell what and why. Metrics tell how often and how fast. Evaluation tells how good.

**Protect -> Observe -> Diagnose -> Measure -> Improve**

### **Final architecture sentence**

User request -> Runtime and model -> tool arguments -> Gateway authentication  
-> Cedar authorization -> Guardrail content check -> Lambda if permitted ->  
CloudWatch telemetry -> Evaluation quality score.